

CSCE 465 - Homework 2

Protect Agent Messages with Classic Cryptography

Jared White

<https://github.com/Jared-04/CSCE-465-HW2>

Task 1 - Demonstrate Insufficient Cryptography

In the first part, we demonstrate that encryption alone is not sufficient protection against attacks. We will show this by simulating the encryption and sending of the command:

```
{"action": "READ", "path": "notes.txt"}
```

In this scenario, User 1 attempts to send the command to User 2 by encrypting it with AES-CTR using a shared key and IV. The attacker is aware of the message and encryption standard, but does not know the key used to encrypt the message. We will apply two attacks; a replay and relay attack.

For the replay, the attacker captures a valid ciphertext from User 1. From there, they can resend it to User 2 at any time. Because a static IV is used throughout communication, the decryption mechanism is identical every time. This means a piece of ciphertext will decrypt identically every time, which was demonstrated in the experiment. Without having unique nonces, counters/sequences, or timestamps implemented into the session, the system will blindly treat every piece of ciphertext the exact same, making duplication very simple.

For the relay attack, we use the same setup of an attacker intercepting a message from User 1 to User 2. Instead of replaying it, the attacker will modify the ciphertext so that User 2 decrypts it to a desired plaintext whilst looking entirely legitimate. In our experiment, we had the attacker change "READ" to "COPY". This was accomplished by calculating the delta XOR of the characters in READ and COPY. This mask can be applied to the captured ciphertext. Because of the properties of XOR, and the fact that AES-CTR applies a flat key to the plaintext, the text will be modified while still being a completely valid ciphertext for the key used. Since this simple system lacks a system of authentication, the ciphertext comes across as legitimate.

Task 2 - Finite Field Diffie-Hellman

As part of improving our algorithm, we develop a Diffie-Hellman key exchange algorithm. This local handshake is used to create a secure communication channel between a gateway and node. We establish the key by generating two fresh private values, along with a known primitive root and prime, and exchanging them in such a way that we apply the secret values as exponents to create a shared secret. During the exchange, the public keys are compiled into a transcript that can be signed by each user using a public RSA key. The resulting shared secret is then used as a master key to develop directional keys and a session ID for symmetric encryption.

A man-in-the-middle occurs by an attacker acting as a "middle-man" during the exchange. The attacker exchanges their own key with both users and is able to control the flow of data by making each user think they have a shared secret with the other. To prevent this, we sign the identities, roles, nonces, and public keys. If an attacker were to attempt a man-in-the-middle attack, they would not have the correct private RSA key to sign the information. This means they could not forge this data, and each user can authenticate the person sending them the Diffie-Hellman key exchange values. If the signatures were removed, then it would still be possible for the attacker to forge a session with both users.

A reflection attack works in a similar way, where the attacker intercepts a user's message and attempts to send it back to the user in order to create a valid session. This again fails because the attacker would need to sign the message, in which case it would come across as being signed by User 1.

Task 3 - Encrypt-then-MAC Layer

For this task, we flesh out the encrypted session system. We do this with the seal and open_record functions which encrypts the plaintext using an encrypt-then-MAC system. The session implements a sequence counter when it is created, which is applied to the IV so that a unique IV is used each time. Additionally, the header, ciphertext, and IV are all hashed together to create a tag that is attached to the end of the packet for authentication.

Task 4 - Adversarial Tests

The attacker is able to intercept, modify, replay, or reflect data over the network. We work with the assumption that the attacker can not invert secure hash functions (SHA-256), break 3072-bit RSA or Diffie-Hellman values, or compromise anything on the hardware side (only attacks network). Non-goals would be an attacker who has credentials, such as a malicious insider, since they would have authorization to encrypt or decrypt with proper keys.

As stated before, AES-CTR with static IVs is vulnerable to a replay or relay attack since the generated stream cipher would be the exact same every time. To fix this issue, a sequence is added to the session ID to create an initialization vector that is unique. Both session ID and sequence are necessary here. The sequence values increment when sending and receiving data, so they will never be the exact same between messages. Session ID prevents this ciphertext from being replayed in a separate session, while the sequence ensures that a ciphertext isn't replayed within the same session.

Verifying before encryption is important, since decrypting first opens up multiple implementation vulnerabilities. The encrypt-then-MAC system ensures that the message can be verified before ever having to encrypt it, since the hash of the header, IV, and ciphertext must be consistent before attempting to decrypt it. If the tag is valid, it can be assumed that the ciphertext is trusted, which ensures that the receiver is not exposed to unsafe decryption pathways.

If the same key is used for both encryption and MAC, then it can cause confusion that the attacker can exploit. If one key is compromised, then both are now compromised and must be replaced. An attacker can also trick an endpoint into believing a piece of ciphertext is a MAC tag or vice versa.

The AEAD mode is another system designed to bundle encryption and message authentication, similar to encrypt-then-MAC. It has some weaknesses, however, when it comes to nonce reuse. Reusing a GCM compromises the authentication key in this system, while the encrypt-then-MAC system utilizes a stronger sequence-backed IV.

While the methods of cryptography used are helpful for authenticating who sent the data, it does not account for accidental misuse or malicious insider behavior. The ciphertext can pass all the tests it needs to, but still contain malicious code by someone who had proper credentials. That is why it's important to still handle the decrypted data in a responsible manner.